



电子科技大学

University of Electronic Science and Technology of China

信息与软件工程学院

SCHOOL OF INFORMATION AND SOFTWARE ENGINEERING

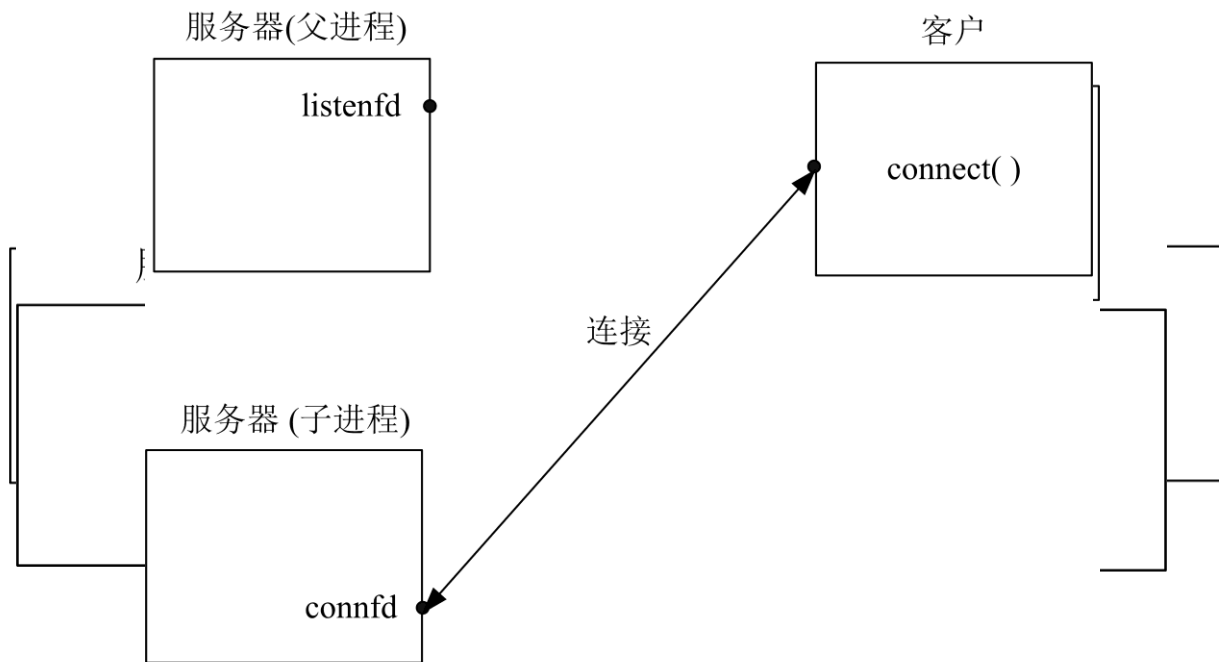
W2D1&W2D2: Linux下的网络 编程基础

Linux下Web Server的设计

多进程并发服务器基本架构

```
pid_t pid;
int listenfd, connfd;
listenfd = Socket( ... );
    /* fill in sockaddr_in{} with server's well-known port */
Bind(listenfd, ... );
Listen(listenfd, LISTENQ);
for ( ; ; ) {
    connfd = Accept (listenfd, ... ); /* probably blocks */
    if( (pid = Fork()) == 0 ) {
        Close(listenfd); /* child closes listening socket */
        doit(connfd);    /* process the request */
        Close(connfd);   /* done with this client */
        exit(0);         /* child terminates */
    }
    Close(connfd);      /* parent closes connected socket */
}
```

多进程并发服务器执行状态



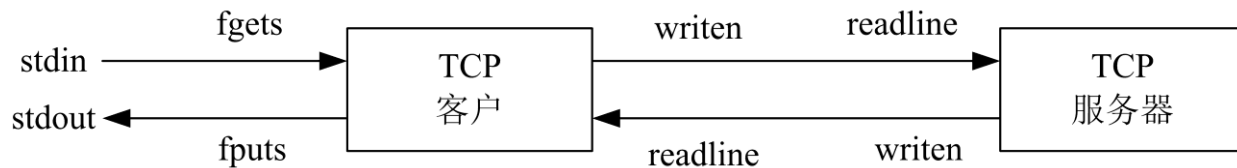
父子进程关闭相应套接口后客户-服务器的状态



主要内容

- 概述
- TCP回射服务器/客户程序
- 正常启动与终止
- Posix信号处理
- 处理SIGCHLD信号
- wait和waitpid函数
- 程序运行异常处理
- 数据格式
- 小结





简单的回射客户-服务器

- 1) 客户从标准输入读一行文本，写到服务器上；
- 2) 服务器从网络输入读此行，并回射给客户；
- 3) 客户读此回射行并写到标准输出。

二、TCP回射服务器/客户程序

服务器程序:

Socket, Bind, Listen, Accept, Fork, **str_echo**, Close

str_echo函数:

Readline, Writen

客户程序:

Socket, Inet pton, Connect, **str cli**

str_cli函数:

Fgets, Writen, Readline, Fputs

服务器启动后，调用socket、bind、listen和accept，并阻塞于accept调用（还没有启动客户）。

三、正常启动与终止

客户调用socket 和connect（引起TCP的三次握手过程）：

当三次握手完成后，connect返回客户，accept 返回服务器，连接建立，接着执行以下步骤：

1) 客户调用str_cli函数，该函数将阻塞于fgets调用，因为我们还从未输入过一行文本。

2) 当accept返回服务器后，服务器调用fork。由子进程调用str_echo。此函数调用readline，它又因调用read而阻塞，等待客户送出一行。

3) 另一方面，服务器父进程再次调用accept 阻塞，等待下一个客户连接。

至此，我们三个进程，它们都在睡眠（阻塞）：客户、服务器父进程和服务子进程。



三、正常启动与终止

在标准输入上输入，得到回射。键入EOF字符（Ctrl+D）以终止客户

1) 当我们键入EOF（Ctrl+D）字符时，fgets返回一个空指针，于是str_cli返回。

2) 当str_cli返回到客户的函数main时，main通过调用exit终止。

3) 进程终止处理的一部分是关闭所有打开的描述字，所以客户套接口由内核关闭。这导致客户TCP发送一个FIN给服务器，服务器TCP则以ACK响应，这就是TCP连接终止序列的前半部分。

4) 当服务器TCP接收FIN时，服务器子进程阻塞于readline调用，于是readline返回0，这导致函数str_echo返回服务器子进程的main。



三、正常启动与终止

5) 服务器子进程通过调用exit来终止。

6) 服务器子进程中打开的所有描述符随之关闭。由子进程来关闭已连接套接口引发TCP连接终止序列的最后两个分节。至此，连接完全终止，客户套接口进入TIME_WAIT状态。

7) 进程终止处理的另一部分是在服务器子进程终止时给父进程发一个信号SIGCHLD。这在本例中发生了，但我们在代码中未捕获这个信号，而这个信号的缺省动作是忽略的。

现在，子进程的状态是Z（僵尸）



四、Posix信号处理

信号是发生某事件时对进程的通知，有时称为软中断。它一般是异步的，这就是说，进程不可能提前知道信号发生的时间。

信号可以：

- 1) 由一进程发往另一进程（或本身）；
- 2) 由内核发往某进程。

SIGCHLD就是内核在某进程终止时发给此进程的父进程的信号。

建立信号的处理办法的Posix方法就是调用函数sigaction。不过这有点复杂，因为该函数的一个参数是我们必须分配并填写的结构。简单些的方法就是调用函数signal，其第一个参数是信号名，第二个参数或为指向函数的指针，或为常值 SIG_IGN或SIG_DFL。



五、处理SIGCHLD信号

设置僵尸 (zombie) 状态的目的就是维护子进程的信息，以便父进程在稍后的某个时候取回。此信息包括子进程的ID、终止状态以及子进程的资源利用信息 (CPU时间、内存等等)。

如果一个进程终止，且该进程有子进程处于僵尸状态，则所有僵尸子进程的父进程ID均置为1 (init进程)。

init进程将作为这些子进程的继父并负责清除它们 (也就是说，**init进程将wait它们，从而去除僵尸进程**)。



五、处理SIGCHLD信号

僵尸进程占用内核空间，最终导致我们无法正常工作。

无论何时我们创建子进程都必须等待，以免变成僵尸进程。

为此，我们需要建立一个信号处理程序来捕获信号SIGCHLD，在处理程序中我们调用wait。

调用Listen 之后，增加函数调用：

Signal (SIGCHLD , sig_chld) ;



五、处理SIGCHLD信号

处理SIGCHLD信号的流程

- 1) 键入EOF字符来终止客户，客户TCP发一个FIN给服务器，服务器以ACK响应。
- 2) FIN的接收导致服务器TCP递送一个EOF给子进程阻塞中的readline，从而子进程终止。
- 3) 当信号SIGCHLD递交时，父进程阻塞于accept调用。函数sig_chld（信号处理程序）执行，它调用函数wait取到子进程的PID和终止状态，再调用 printf，然后返回。
- 4) 由于该信号是在父进程阻塞于慢系统调用（accept）时由父进程捕获的，所以内核将使accept返回一个EINTR错误（被中断的系统调用），而父进程不处理此错误，所以中止。

五、处理SIGCHLD信号

```
for ( ; ; ) {  
    clilen = sizeof (cliaddr);  
    if ( (connfd = accept (listenfd, (SA *) &cliaddr, &clilen)) < 0)  
{  
        if (errno == EINTR)  
            continue;      /* back to for () */  
        else  
            err_sys ("accept error");  
    }  
}
```



六、wait和waitpid函数

如果没有终止的子进程让进程来调用wait，但有一个或多个正在执行的子进程，则wait阻塞直到第一个现有子进程终止

```
#include <sys/wait.h>
```

```
pid_t wait(int * statloc);
```

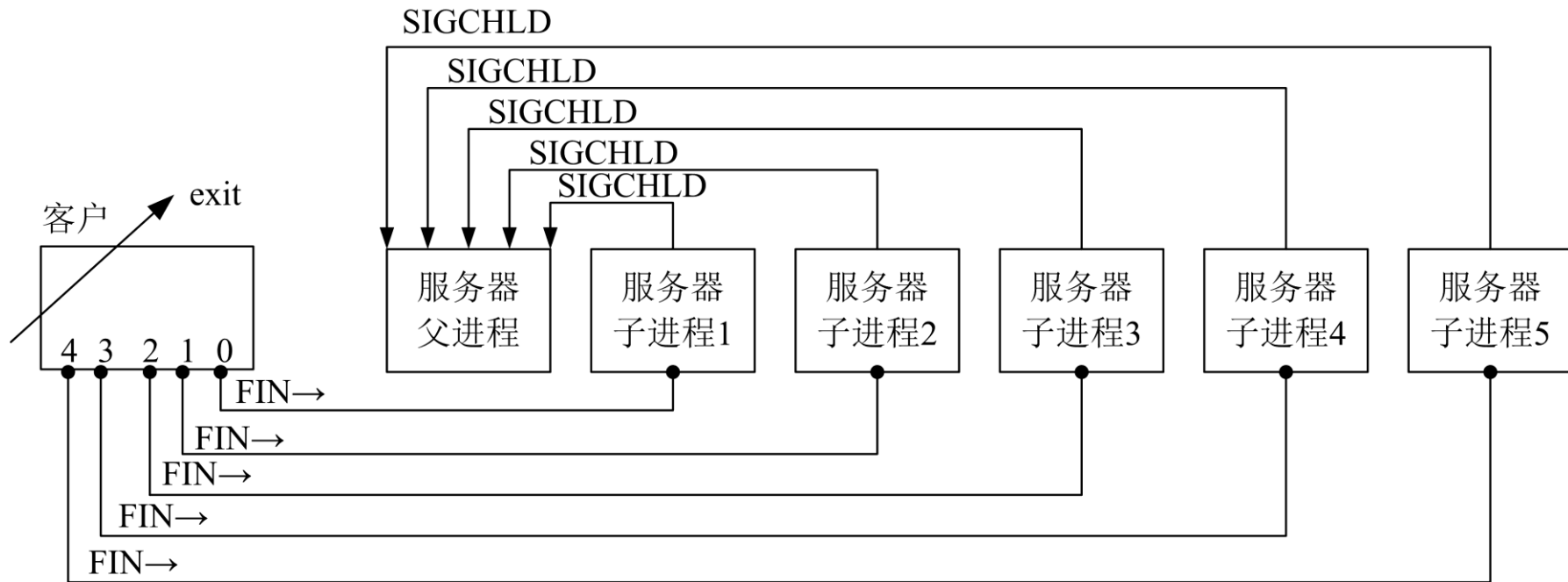
```
pid_t waitpid(pid_t pid,int * statloc,int options);
```

二者均返回：子进程**PID**——成功；或为**-1**——出错

- 参数 pid指定想等待的进程ID，值-1表示等待第一个终止的子进程。（也有其他选项，它们涉及到进程组ID）
- 参数options指定附加选项，最常用的选项是WNOHANG，它通知内核在沒有已终止子进程时不要阻塞。



六、wait和waitpid函数



客户终止导致关闭5个连接，终止5个子进程



六、wait和waitpid函数

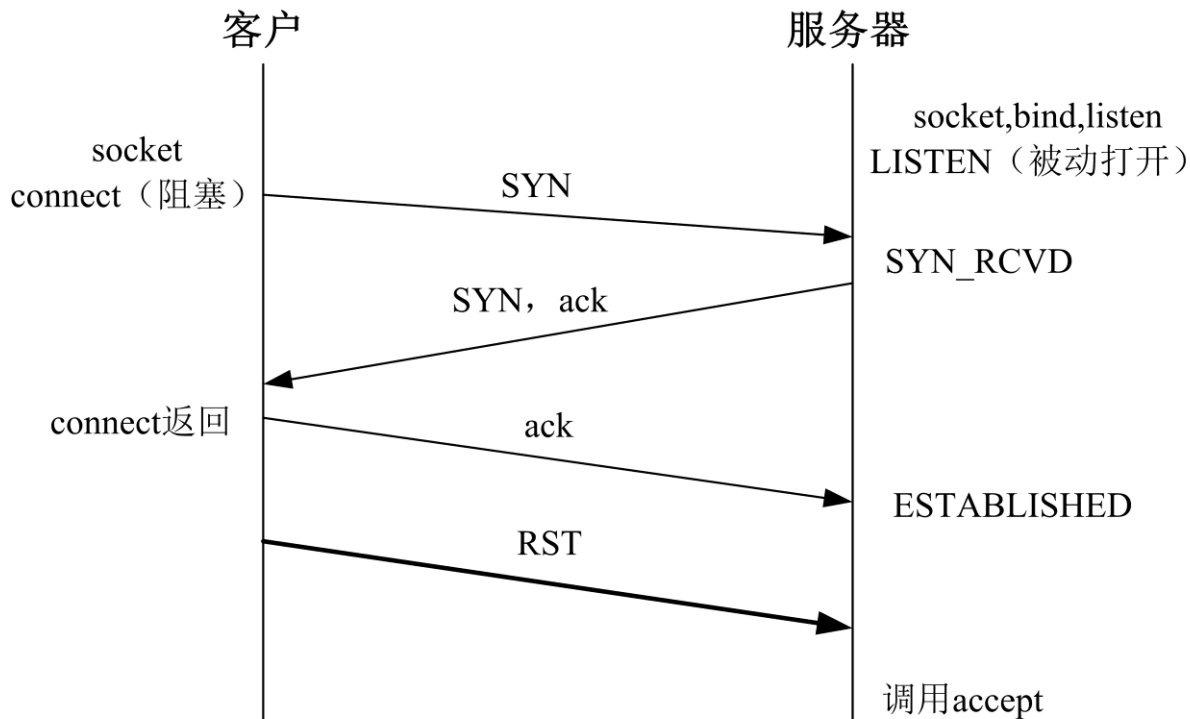
```
#include      "unp.h"

void
sig_chld(int signo){
    pid_t    pid;
    int      stat;
    while ( (pid = waitpid(-1, &stat, WNOHANG)) > 0)
        printf("child %d terminated\n", pid);
    return;
}
```



七、程序运行异常处理

1. Accept返回前连接夭折



ESTABLISHED状态的连接在调用accept之前收到RST



七、程序运行异常处理

2. 服务器进程终止

服务器子进程终止（杀死该进程），作为进程终止处理的部分工作，子进程打开的描述字都关闭，这引发了一个FIN发送给客户，客户TCP相应地以ACK响应。这就是TCP连接终止的前一半工作。

当FIN到达套接口时，客户阻塞于fgets调用。客户工作时有两个描述字——套接口和用户输入，它不能只阻塞于两个源中的某个特定源输入，而是应阻塞于任一源的输入。



七、程序运行异常处理

3. SIGPIPE信号

如果客户不理睬函数readline返回的错误而写入更多的数据到服务器上，又会怎样？这种情况是可能发生的，譬如，客户在读回任何东西之前对服务器写两次，而第一次写就引发了一RST。

规则：当一个进程向接收了RST的套接口进行写操作时，内核给该进程发一个SIGPIPE信号。此信号的默认行为是终止进程，所以，进程必须捕获它以避免不情愿地被终止。

进程不论是捕获了该信号并从其信号处理程序返回，还是不理睬该信号，写操作都返回EPIPE错误。



七、程序运行异常处理

4. 服务器主机崩溃

5. 服务器主机崩溃后重启

6. 服务器主机关机



八、数据格式

问题：

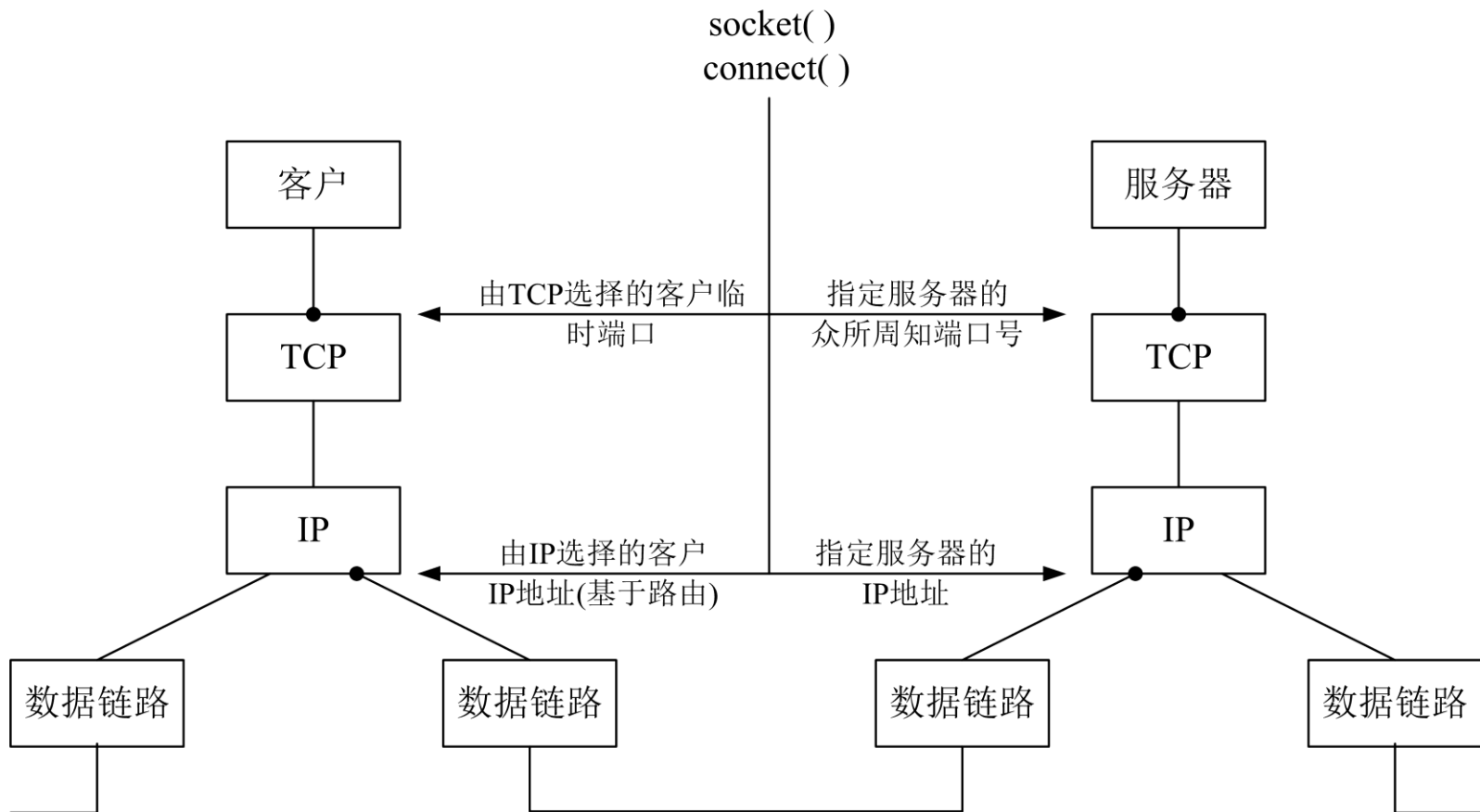
客户端从标准输入读入两个用空格隔开的整数，服务器端返回两个整数之和。

- 方法一 在客户与服务器之间传递文本串
- 方法二 在客户与服务器端传递二进制结构

方法一
OK!
方法二问
题多多～

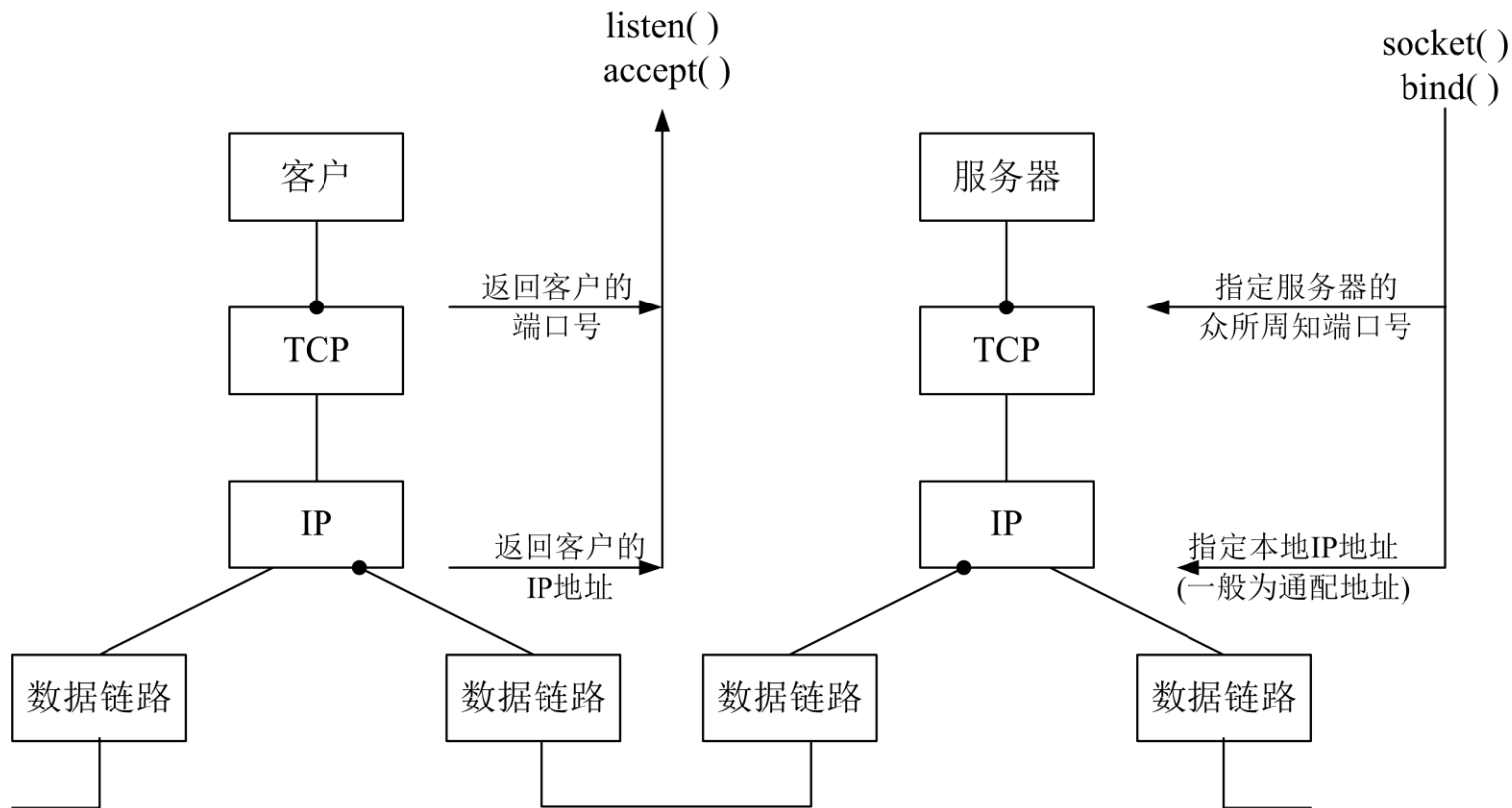


小结



从客户的角度总结TCP客户-服务器

小结



从服务器的角度总结TCP客户-服务器



小结

最基础的文本客户端多进程服务器留下了待检查的许多细节问题

僵尸进程问题：通过捕获信号SIGCHLD来处理。接着，信号处理程序调用waitpid，已证明必须调用此函数而不是早期的wait函数，因为Unix信号是不排队的。

问题二：当服务器进程终止时，客户不知道。客户的TCP是被通知了，但客户本身由于正阻塞于等待用户输入而未接收到此通知。

后续会用函数select和poll来处理这种情况（Day9-10），它等待多个描述字中的任何一个准备好即可，而不是阻塞于单个描述字。



小结

如果服务器主机崩溃，也要等到客户向该服务器发送了数据才能检测到。一些应用进程必须尽早意识到这个事实，
后续可考虑套接口选项SO_KEEPALIVE。

这两天设计的TCP服务器的基本要求是交换文本行，由于服务器不检查它回射的行，所以它工作得很好，
但在客户与服务器之间发送数值数据时将引发新问题。



训练1

- 训练内容-开发面向多客户连接的TCP文本聊天程序

文本聊天程序的多方分别是服务器和多个客户端（服务器和多个客户端分别运行在不同的Ubuntu虚拟机中）

1.服务器

服务器程序等待某个客户端的连接，客户连接成功后，服务器通过创建子进程来与客户进行文字通信（显示通信内容），子进程如果收到客户方的exit文本，则退出子进程。父进程则继续等待其他客户端的连接。

2.客户端

建立一个用TCP连接的客户，以命令行的方式启动，命令行要求有服务器地址，服务器端口号。连接建立成功后可以输入文本与服务器进行通信（显示通信内容）。客户端通过输入exit文本结束与服务器的通信并退出程序。



Webserver V0.7版

网络服务模块-多进程版



mini_webserver

工程基础模块 Day1 && Day4

- 构建子模块 D1
Makefile
编译、清理、调试版本
- 配置子模块 D1
Config.c
端口、根目录、线程数
- 日志子模块 D1,D4
log.c
访问日志、错误日志

用户数据模块 Day2 && Day3

- 用户结构子模块 D2-D3
user.c
链表、二叉树、增删改查
- CSV存储子模块 D2
csv_store.c
用户数据加载和保存
- 索引查找子模块 D3
user_index.c
按用户名快速查找

网络服务模块 Day6 && Day7

- TCP监听子模块 D6
server.c
socket, bind, listen
- 连接管理子模块 D7
client.c
连接与状态、关闭释放

并发处理模块 Day4-5 && Day8-9

- 进程模型子模块 D4
process.c
父子进程协同
- 线程同步子模块 D5
thread.c
锁、条件变量、共享资源保护
- 线程池子模块 D8
thread_pool.c
任务队列、工作线程
- 事件驱动子模块 D9
event_loop.c
select / epoll

HTTP业务模块 Day10-Day14

- 请求解析子模块 D10
http_request.c
方法、URL、Header、Body
- 静态资源子模块 D11
static_file.c
HTML、CSS、图片读取
- GET/POST处理子模块 D12
handler.c
表单提交、参数读取
- 路由配置子模块 D13
router.c
URL匹配、处理函数绑定
- 认证安全子模块 D14
auth.c
登录校验、权限控制、路径检查
- 响应生成子模块 D10-D14
http_response.c
状态码、响应头、响应体

测试模块 Day1-15

- 每日增加功能的相关测试
如: test_day01.sh
- 如: bench.c 并发压力测试/响应时间等

Web Server V0.7开始，服务器需要能够“同时”处理来自多方的HTTP请求——使用多进程处理HTTP请求

多个客户端客户端发 HTTP 请求，服务器解析路径并返回 HTTP 响应

```
curl http://127.0.0.1:8080/hello  
curl http://127.0.0.1:8080/users/zhangsan  
curl http://127.0.0.1:8080/not-found
```

使用curl命令模拟http请求:

```
curl http://127.0.0.1:8080/hello
```

CURL的官方定义为:curl is a command line tool for transferring data with URL syntax,即使用URL语法规则来传输数据的命令行工具。



浏览器或 curl 发来的内容类似:

```
GET /hello HTTP/1.1  
Host: 127.0.0.1:8080  
User-Agent: curl/...
```

本次先只处理第一行

除了多进程处理请求之外，还需要满足以下处理需求：

1. 前面提到如果父进程不回收子进程，就会产生僵尸进程。

```
void sig_chld(int signo) {  
    int stat;  
    pid_t pid;  
    while ((pid = waitpid(-1, &stat, WNOHANG)) > 0) {  
        printf("child %d terminated\n", pid);  
    }  
}
```

```
signal(SIGCHLD, sig_chld);
```



2. accept 被 EINTR 打断打处理的方式

```
connfd = accept(listenfd, NULL, NULL);
if (connfd < 0) {
    if (errno == EINTR) {
        continue;
    }
    perror("accept");
    break;
}
```



除了多进程处理请求之外，还需要满足以下处理需求：

3. SIGPIPE 和客户端异常断开

- ① 客户端连接后立刻关闭。
- ② 服务器仍然 send 响应。
- ③ 系统可能发送 SIGPIPE，导致服务器进程终止。

```
signal(SIGPIPE, SIG_IGN);
```

因此V0.7的多进程并发服务器要求：父进程持续接收客户端连接，每个客户端交给一个子进程处理。父进程要处理 SIGCHLD，避免僵尸进程；同时要处理 accept 被信号打断的问题。

新增文件包括：

- include/tcp_fork_server.h
- src/tcp_fork_server.c
- tests/test_day07.sh

修改文件包括

- src/main.c
- include/request_handler.h
- src/request_handler.c
- Makefile
- README.md
- docs/debug-log.txt (可选)

V0.7版 mini_webserver-功能要求

- 父进程创建监听socket
- 父进程循环accept
- 每个客户端连接创建一个子进程
- 子进程处理HTTP请求并返回响应
- 父进程关闭自己的connfd
- 子进程关闭继承来的listenfd
- 子进程处理完请求后exit
- 父进程捕获SIGCHLD使用waitpid(-1, &stat, WNOHANG) 回收子进程
- accept遇到EINTR时继续等待
- 忽略或处理SIGPIPE
- 处理到max_clients后退出，方便自动测试



V0.7版 mini_webserver

```
for i in 1 2 3 4 5; do  
    curl -s  
    http://127.0.0.1:8080/hello &  
done  
wait
```

测试bash示意

